

1.数组G[1024][768]数据类型为Uint8(无符号单字节整型), 已存放1024x768像素的图片灰度。

计算:1)最小灰度阈值Gav, 使得大于Gav的像素点数不超过50%;

2)大于Gav的像素总数量N;

3)所有像素平均灰度值Avg。

要求:循环体内须尽可能使用最经济类型的变量, 并使用合理的计算统计方法, 以缩短代码运行时间。

阶段一：单次遍历图像 数组

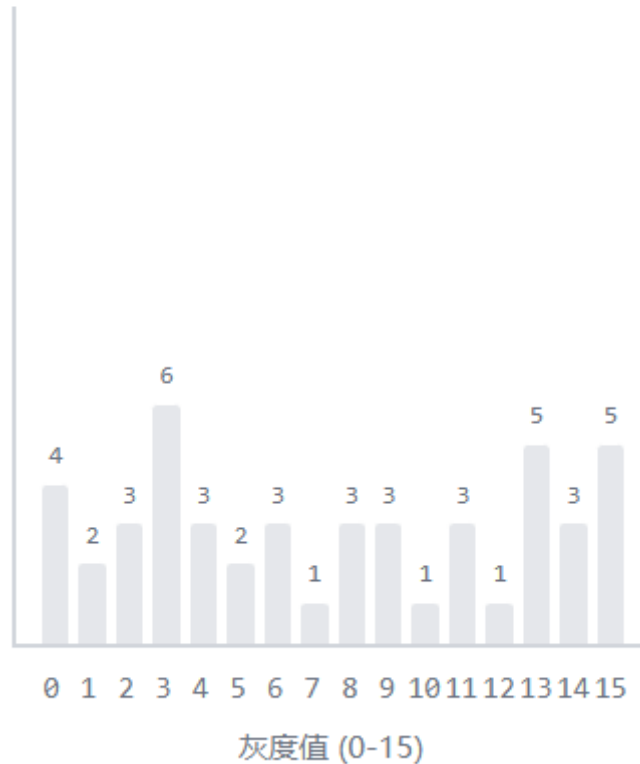
```
hist[G[i]]++
```

12	0	8	5	11	13	13	4	9	3	11	1
15	15	13	11	5	13	3	4	0	4	2	14
8	6	14	9	8	3	6	3	7	3	2	2
1	15	9	13	0	0	6	15	3	15	10	14

当前指针 **i**: 47

当前灰度 **G[i]**: 14

直方图 (Histogram)



```
#include <stdint.h>
#include <stdio.h>

#define PIXEL_COUNT 786432 // 1024 * 768
#define HALF_COUNT 393216 // 786432 / 2

// 假设 G 数组的指针已经通过某种方式获取
void calculate_image_statistics(const uint8_t *G) {

    // --- 第一阶段：合理统计方法（直方图单次遍历） ---

    // hist 数组用于记录 0-255 每种灰度的像素个数
    // 某一种灰度最多可能占据全图(786432)，超出 uint16_t(65535)，所以必须用 uint32_t
    uint32_t hist[256] = {0};

    // 最耗时的全图遍历循环。
    // 循环变量 i 最大值 786431，所以采用 uint32_t 最经济。
    // 循环体内仅有一次数组寻址和一次自增运算，没有任何复杂的加减乘除或分支判断。
    for (uint32_t i = 0; i < PIXEL_COUNT; i++) {
        hist[G[i]]++;
    }

    // --- 第二阶段：极简计算 ---

    // 1. 计算所有像素平均灰度值 Avg
```

```

// 即使所有像素都是 255，总和最大为 786432 * 255 = 200,540,160
// 这个数值远未超过 uint32_t 的极值(42.9亿)，所以 sum 用 uint32_t 即可，无需 64 位整
型！
uint32_t sum = 0;
for (uint16_t i = 0; i < 256; i++) {
    sum += i * hist[i];
}

// 如果硬件没有 FPU（浮点运算单元），整数除法最快。如果需要精度，可强转 float
uint8_t Avg_int = sum / PIXEL_COUNT;
float Avg_float = (float)sum / PIXEL_COUNT;

// 2. 计算 Gav 和 N
uint32_t N = 0;
uint8_t Gav = 0;

// 从最高灰度(255)向低灰度(0)累加，寻找满足“不超过50%”的最小阈值
// i 使用 uint8_t 会在 i=0 时递减变成 255 导致死循环，所以这里用 int16_t
for (int16_t i = 255; i >= 0; i--) {
    if (N + hist[i] <= HALF_COUNT) {
        N += hist[i]; // 累加当前灰度值的像素数
    } else {
        // 如果加上当前的像素数就超过了 50%
        // 说明当前灰度值 i 必须作为阈值 Gav（即只有严格 > i 的像素才算入 N）
        Gav = i;
        break; // 找到最小阈值，立刻跳出循环
    }
}

printf("1) 最小灰度阈值 Gav: %u\n", Gav);
printf("2) 大于 Gav 的像素总数量 N: %u\n", N);
printf("3) 所有像素平均灰度值 Avg（整数/浮点）: %u / %.2f\n", Avg_int,
Avg_float);
}

int main(void)
{
    uint8_t G[PIXEL_COUNT];
    // 初始化测试数据（实际使用时替换为真实图像数据）
    for (uint32_t i = 0; i < PIXEL_COUNT; i++) {
        G[i] = (uint8_t)(i % 256);
    }
    calculate_image_statistics(G);
    return 0;
}

```

2. 已知 $x < 1$ 且非负值。为缩短运算时间采取以下计算方法代替调用内部数学函数 $Y = \exp(x)$ (其中 x 为单精度变量):

- 1) 查表常量数组 $Y[256]$ ，其中可以预编译或初始化方式对该数组赋值;
- 2) 对查表计算出的 Y 值进行修正(如线性修正或高次修正)。



```
#include <stdio.h>
#include <math.h>
#include <stdint.h>
```

// 预编译常量数组: Y[i] = exp(i/255), 覆盖 [0, 1] 区间

```
static const float Y[256] = {
    1.000000f, 1.003930f, 1.007875f, 1.011836f, 1.015813f, 1.019806f, 1.023815f,
    1.027840f,
    1.031881f, 1.035938f, 1.040012f, 1.044102f, 1.048208f, 1.052331f, 1.056471f,
    1.060627f,
    1.064800f, 1.068990f, 1.073197f, 1.077421f, 1.081662f, 1.085920f, 1.090196f,
    1.094489f,
    1.098799f, 1.103127f, 1.107473f, 1.111836f, 1.116217f, 1.120616f, 1.125033f,
    1.129468f,
    1.133921f, 1.138392f, 1.142882f, 1.147390f, 1.151916f, 1.156461f, 1.161025f,
    1.165607f,
    1.170208f, 1.174828f, 1.179467f, 1.184125f, 1.188802f, 1.193498f, 1.198214f,
    1.202949f,
    1.207703f, 1.212477f, 1.217270f, 1.222083f, 1.226916f, 1.231768f, 1.236641f,
    1.241533f,
    1.246445f, 1.251378f, 1.256330f, 1.261303f, 1.266296f, 1.271309f, 1.276343f,
    1.281397f,
    1.286472f, 1.291568f, 1.296684f, 1.301821f, 1.306979f, 1.312158f, 1.317358f,
    1.322580f,
    1.327822f, 1.333086f, 1.338371f, 1.343678f, 1.349006f, 1.354356f, 1.359727f,
    1.365120f,
    1.370535f, 1.375972f, 1.381431f, 1.386912f, 1.392415f, 1.397941f, 1.403489f,
    1.409059f,
    1.414652f, 1.420267f, 1.425905f, 1.431566f, 1.437250f, 1.442957f, 1.448687f,
    1.454440f,
    1.460216f, 1.466016f, 1.471839f, 1.477685f, 1.483555f, 1.489449f, 1.495366f,
    1.501307f,
    1.507272f, 1.513261f, 1.519274f, 1.525311f, 1.531372f, 1.537457f, 1.543567f,
    1.549702f,
    1.555861f, 1.562044f, 1.568253f, 1.574486f, 1.580744f, 1.587027f, 1.593335f,
    1.599669f,
```

```

    1.606028f, 1.612412f, 1.618822f, 1.625257f, 1.631718f, 1.638205f, 1.644718f,
    1.651257f,
    1.657822f, 1.664413f, 1.671031f, 1.677675f, 1.684345f, 1.691042f, 1.697766f,
    1.704517f,
    1.711294f, 1.718099f, 1.724931f, 1.731790f, 1.738677f, 1.745591f, 1.752533f,
    1.759503f,
    1.766501f, 1.773527f, 1.780581f, 1.787664f, 1.794775f, 1.801915f, 1.809083f,
    1.816281f,
    1.823507f, 1.830763f, 1.838048f, 1.845363f, 1.852707f, 1.860081f, 1.867485f,
    1.874919f,
    1.882383f, 1.889877f, 1.897402f, 1.904957f, 1.912543f, 1.920160f, 1.927808f,
    1.935487f,
    1.943197f, 1.950939f, 1.958712f, 1.966517f, 1.974354f, 1.982223f, 1.990124f,
    1.998058f,
    2.006024f, 2.014023f, 2.022054f, 2.030119f, 2.038217f, 2.046348f, 2.054513f,
    2.062711f,
    2.070943f, 2.079209f, 2.087509f, 2.095843f, 2.104212f, 2.112615f, 2.121053f,
    2.129526f,
    2.138034f, 2.146577f, 2.155155f, 2.163769f, 2.172419f, 2.181104f, 2.189826f,
    2.198584f,
    2.207378f, 2.216209f, 2.225076f, 2.233980f, 2.242921f, 2.251900f, 2.260916f,
    2.269969f,
    2.279060f, 2.288189f, 2.297356f, 2.306562f, 2.315806f, 2.325089f, 2.334411f,
    2.343772f,
    2.353172f, 2.362612f, 2.372091f, 2.381611f, 2.391170f, 2.400770f, 2.410410f,
    2.420091f,
    2.429813f, 2.439576f, 2.449380f, 2.459226f, 2.469113f, 2.479043f, 2.489014f,
    2.499028f,
    2.509084f, 2.519183f, 2.529325f, 2.539510f, 2.549739f, 2.560011f, 2.570327f,
    2.580687f,
    2.591091f, 2.601540f, 2.612033f, 2.622571f, 2.633154f, 2.643783f, 2.654457f,
    2.665177f,
    2.675943f, 2.686755f, 2.697614f, 2.708520f, 2.719473f, 2.730473f, 2.741521f,
    2.752617f,
};

```

// 方法1: 线性修正 (线性插值)

// 用两个点连直线近似曲线, 运算量: 1次乘法 + 1次加法

float exp_linear(float x)

```

{
    float idx_f = x * 255.0f;
    int32_t i = (int32_t)idx_f;
    float f = idx_f - (float)i;

    // result = Y[i] + f × (Y[i+1] - Y[i])
    return Y[i] + f * (Y[i + 1] - Y[i]);
}

```

// 方法2: 高次修正 (二次插值 / 抛物线插值)

// 用三个点拟合抛物线, 更贴合 exp 曲线, 运算量: 2次乘法 + 2次加法

// 公式: Newton 向前差分插值

// result = Y[i] + f×ΔY[i] + f×(f-1)/2 × Δ²Y[i]

// 其中 ΔY[i] = Y[i+1] - Y[i] (一阶差分)

// Δ²Y[i] = Y[i+2] - 2×Y[i+1] + Y[i] (二阶差分)

float exp_quadratic(float x)

```

{
    float idx_f = x * 255.0f;
    int32_t i = (int32_t)idx_f;

```

```

float f = idx_f - (float)i;

float d1 = Y[i + 1] - Y[i];           // 一阶差分
float d2 = Y[i + 2] - 2.0f * Y[i + 1] + Y[i]; // 二阶差分

return Y[i] + f * d1 + f * (f - 1.0f) * 0.5f * d2;
}

int main(void)
{
    float test_values[] = {0.0f, 0.1f, 0.25f, 0.5f, 0.75f, 0.9f, 0.99f};
    int n = sizeof(test_values) / sizeof(test_values[0]);

    printf("   x   | exp(x)   | 线性修正   | 误差       | 二次修正   | 误差\n");
    printf("-----|-----|-----|-----|-----|-----\n");

    for (int j = 0; j < n; j++) {
        float x = test_values[j];
        float exact = expf(x);
        float lin = exp_linear(x);
        float quad = exp_quadratic(x);
        printf(" %.2f   | %.6f | %.6f   | %+.6f   | %.6f   | %+.6f\n",
            x, exact, lin, lin - exact, quad, quad - exact);
    }

    return 0;
}

```

3. 设异步串行发送数据的波特率为9600，8bit无校验方式。接收端用某一GPIO作为RXD。请使用软件模拟：

- 1) 在RXD起始下降沿的中断响应开始，使用定时器触发逐位接收一个单字节数据帧；
- 2) 将每次接收的字节数据，存入自定义指针指向的FIFO缓冲区。

软件 UART 状态机时序演示 (8N1, 9600bps)

数据帧: 0xA5 (10100101) | 发送顺序: LSB优先 (1->0->1->0->0->1->0->1)

再次触发接收

复位

RXD 引脚波形与采样时序

采样触发点 1 Bit = 104μs



软件中断流转 (ISR)

RXD_FALLING_EDGE_IRQ()

关闭边沿检测

TIMER_START(52μs)

TIMER_IRQ() 状态机

STOP, 开启边沿检测

STATE_START_BIT
完成

STATE_DATA_BITS
8位收齐

STATE_STOP_BIT
空闲

数据组装 (rx_byte)

rx_byte |= (pin << index)

D7 D6 D5 D4 D3 D2 D1 D0
1 0 1 0 0 1 0 1

当前位指针 (rx_bit_index): 7

Hex: 0xA5

FIFO 缓冲区 (rx_fifo)

0xA5

```
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>

// =====
// 硬件抽象层（实际项目中替换为真实寄存器操作）
// =====
#define GPIO_READ(pin)      0    // 读取GPIO电平，返回0或1
#define TIMER_START(us)     ((void)(us)) // 启动定时器，us微秒后触发中断
#define TIMER_STOP()        ((void)0)    // 停止定时器
#define ENABLE_FALLING_EDGE() ((void)0)  // 使能下降沿中断
#define DISABLE_FALLING_EDGE() ((void)0)  // 禁止下降沿中断

// =====
// 常量定义
// =====
#define BAUD_RATE          9600
#define BIT_TIME_US        (1000000 / BAUD_RATE) // 104μs，每位的时间
#define HALF_BIT_US        (BIT_TIME_US / 2)     // 52μs，半位时间（用于采到中间）

// =====
// FIFO 缓冲区
// =====
#define FIFO_SIZE 256 // 必须是2的幂，方便取模

typedef struct {
    uint8_t buf[FIFO_SIZE];
    volatile uint16_t head; // 写入位置
    volatile uint16_t tail; // 读取位置
} fifo_t;

static fifo_t rx_fifo;
```

```

// 写入一字节到FIFO
static void fifo_put(uint8_t byte)
{
    uint16_t next = (rx_fifo.head + 1) & (FIFO_SIZE - 1);
    if (next != rx_fifo.tail) { // 未满才写入
        rx_fifo.buf[rx_fifo.head] = byte;
        rx_fifo.head = next;
    }
}

// 从FIFO读取一字节
static bool fifo_get(uint8_t *byte)
{
    if (rx_fifo.head == rx_fifo.tail) return false; // 空
    *byte = rx_fifo.buf[rx_fifo.tail];
    rx_fifo.tail = (rx_fifo.tail + 1) & (FIFO_SIZE - 1);
    return true;
}

// =====
// 软件UART接收状态机
// =====
typedef enum {
    STATE_IDLE,           // 空闲，等待起始位
    STATE_START_BIT,      // 已检测到下降沿，等待半位后确认起始位
    STATE_DATA_BITS,      // 正在接收数据位
    STATE_STOP_BIT        // 等待停止位
} rx_state_t;

static volatile rx_state_t rx_state = STATE_IDLE;
static volatile uint8_t rx_bit_index; // 当前接收第几位 (0~7)
static volatile uint8_t rx_byte;      // 正在组装的字节

// =====
// 1) 下降沿中断：检测到起始位
// =====
// 当RXD引脚出现下降沿时，硬件触发此中断
void RXD_FALLING_EDGE_IRQ(void)
{
    // 禁止下降沿中断，防止重复触发
    DISABLE_FALLING_EDGE();

    // 启动定时器，等待 半个bit时间 后进入中断
    // 目的：跳到起始位的中间位置采样，确认是否真的是低电平
    rx_state = STATE_START_BIT;
    TIMER_START(HALF_BIT_US);
}

// =====
// 2) 定时器中断：逐位采样
// =====
// 每隔 BIT_TIME_US (104μs) 触发一次，对应每一位的中心
void TIMER_IRQ(void)
{
    uint8_t pin_val = GPIO_READ(0); // 读取RXD引脚电平

    switch (rx_state) {

```



```

case STATE_START_BIT:
    // 在起始位中间采样：应该是低电平(0)
    if (pin_val == 0) {
        // 确认是有效的起始位，开始接收8位数据
        rx_bit_index = 0;
        rx_byte = 0;
        rx_state = STATE_DATA_BITS;
        TIMER_START(BIT_TIME_US); // 等一个bit时间后采第一位
    } else {
        // 假起始位（毛刺），恢复空闲
        rx_state = STATE_IDLE;
        TIMER_STOP();
        ENABLE_FALLING_EDGE();
    }
    break;

case STATE_DATA_BITS:
    // 在数据位中间采样，LSB先收
    //           第0位   第1位   ...   第7位
    // pin_val:  D0     D1     ...    D7
    rx_byte |= (pin_val << rx_bit_index);
    rx_bit_index++;

    if (rx_bit_index >= 8) {
        // 8位收完，下一步等停止位
        rx_state = STATE_STOP_BIT;
    }
    TIMER_START(BIT_TIME_US); // 继续等下一个bit
    break;

case STATE_STOP_BIT:
    // 停止位应该是高电平(1)，这里可以做校验
    // 无论是否正确，都把接收到的字节存入FIFO
    fifo_put(rx_byte);

    // 恢复空闲状态，等待下一个起始位
    rx_state = STATE_IDLE;
    TIMER_STOP();
    ENABLE_FALLING_EDGE(); // 重新使能下降沿检测
    break;

default:
    rx_state = STATE_IDLE;
    TIMER_STOP();
    ENABLE_FALLING_EDGE();
    break;
}
}

// =====
// 测试用主函数
// =====
int main(void)
{
    rx_fifo.head = 0;
    rx_fifo.tail = 0;
    rx_state = STATE_IDLE;

```

```

// 模拟接收几个字节
printf("软UART接收器已初始化\n");
printf("波特率: %d, 格式: 8N1\n", BAUD_RATE);
printf("每位时间: %dμs, 半位时间: %dμs\n", BIT_TIME_US, HALF_BIT_US);

// 模拟接收到字节 0xA5 (10100101)
// 实际使用中, 这些由硬件中断自动触发
rx_byte = 0xA5;
fifo_put(rx_byte);
rx_byte = 0x3C;
fifo_put(rx_byte);

// 从FIFO读取
uint8_t data;
while (fifo_get(&data)) {
    printf("收到: 0x%02X\n", data);
}

return 0;
}

```

4 某一内置电池供电的烟雾报警器的软件代码, 采取每分钟定时一次采样计算得到烟雾浓度 $X(t)$, 并立即无线上传测量数据。实测电池续航约6个月。在硬件配置无法改变的前提下如何优化软件, 请给出你的方案思路:

- 1)如何进一步降低功耗, 延长电池续航时间;
- 2)如何提升报警响应速度, 使得当 $X(t)$ 达到报警值 X_m 后的响应报警延时不超过3秒。



```

#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>

// =====
// 硬件抽象层 (实际项目中替换为真实寄存器操作)
// =====

typedef enum {

```

```

    HW_SLEEP_DEEP,      // 深度休眠（仅ADC比较器工作）
    HW_SLEEP_LIGHT,     // 轻度休眠（定时器保持运行）
    HW_AWAKE            // 完全唤醒
} hw_power_mode_t;

#define ADC_READ()      100      // 读取ADC烟雾浓度值（0~1000）
#define ADC_SET_COMPARATOR(val) ((void)(val)) // 设置ADC比较器唤醒阈值
#define MCU_SLEEP(mode) ((void)(mode)) // MCU进入休眠
#define RTC_WAKEUP_SET(seconds) ((void)(seconds)) // 设置RTC定时唤醒
#define WIRELESS_SEND(data, len) ((void)(data), (void)(len)) // 无线发送
#define ALARM_ON()      ((void)0) // 触发声光报警
#define ALARM_OFF()     ((void)0) // 关闭报警

// =====
// 参数配置
// =====
#define ALARM_THRESHOLD_XM      800    // 报警阈值 Xm
#define WARNING_THRESHOLD_80    640    // 80%阈值，触发快速采样
#define WARNING_THRESHOLD_50    400    // 50%阈值，进入警戒
#define WARNING_THRESHOLD_30    240    // 30%阈值，正常/警戒分界

#define SAMPLE_INTERVAL_NORMAL  60     // 正常采样间隔（秒）
#define SAMPLE_INTERVAL_ALERT   10     // 警戒采样间隔（秒）
#define SAMPLE_INTERVAL_DANGER  3      // 危险采样间隔（秒）

#define UPLOAD_INTERVAL_NORMAL  600    // 正常上传间隔（秒）
#define UPLOAD_INTERVAL_ALERT   60     // 警戒上传间隔（秒）

#define PREDICTION_K            3      // 趋势预测步数
#define TREND_COUNT             3      // 用于计算趋势的历史采样数

// =====
// 运行状态
// =====
typedef enum {
    MODE_NORMAL,      // 正常模式：低频采样、低频上传
    MODE_ALERT,       // 警戒模式：中频采样、中频上传
    MODE_DANGER,      // 危险模式：高频采样、立即上传
    MODE_ALARM        // 报警模式：持续报警
} run_mode_t;

static volatile run_mode_t current_mode = MODE_NORMAL;

// =====
// 采样数据缓存（用于趋势预测和批量上传）
// =====
#define SAMPLE_HISTORY_SIZE 16

typedef struct {
    int16_t values[SAMPLE_HISTORY_SIZE]; // 历史采样值
    uint8_t count;                       // 已存采样数
    uint8_t index;                       // 环形写入位置
} sample_history_t;

static sample_history_t history;

// 存入新采样值
static void history_push(int16_t value)

```

```

{
    history.values[history.index] = value;
    history.index = (history.index + 1) % SAMPLE_HISTORY_SIZE;
    if (history.count < SAMPLE_HISTORY_SIZE) history.count++;
}

// 获取最近的采样值 (0=最新, 1=上一次...)
static int16_t history_get(uint8_t offset)
{
    if (offset >= history.count) return 0;
    uint8_t idx = (history.index - 1 - offset + SAMPLE_HISTORY_SIZE) %
SAMPLE_HISTORY_SIZE;
    return history.values[idx];
}

// =====
// 待上传数据FIFO
// =====
#define UPLOAD_FIFO_SIZE 32

typedef struct {
    int16_t data[UPLOAD_FIFO_SIZE];
    uint8_t count;
} upload_fifo_t;

static upload_fifo_t upload_buf;

static void upload_fifo_push(int16_t value)
{
    if (upload_buf.count < UPLOAD_FIFO_SIZE) {
        upload_buf.data[upload_buf.count++] = value;
    }
}

static void upload_fifo_flush(void)
{
    if (upload_buf.count == 0) return;
    // 打包发送所有缓存数据
    WIRELESS_SEND(upload_buf.data, upload_buf.count * sizeof(int16_t));
    upload_buf.count = 0;
}

// =====
// 趋势预测: 用线性外推预测未来K步的浓度
// =====
// 返回预测值: 当前值 + 变化率 × K
static int16_t predict_future(void)
{
    if (history.count < TREND_COUNT) {
        return history_get(0); // 数据不足, 无法预测
    }

    // 计算平均变化率 (最近 TREND_COUNT 个采样)
    int32_t sum_diff = 0;
    for (uint8_t i = 0; i < TREND_COUNT - 1; i++) {
        sum_diff += (int32_t)history_get(i) - history_get(i + 1);
    }
    int16_t avg_diff = (int16_t)(sum_diff / (TREND_COUNT - 1));

```

```

// 线性外推
int16_t current = history_get(0);
int32_t predicted = (int32_t)current + (int32_t)avg_diff * PREDICTION_K;

// 限幅
if (predicted > 1000) predicted = 1000;
if (predicted < 0) predicted = 0;

return (int16_t)predicted;
}

// =====
// 模式切换：根据当前浓度和预测值决定运行模式
// =====
static void update_mode(int16_t current, int16_t predicted)
{
    // 浓度达到报警值 → 报警模式
    if (current >= ALARM_THRESHOLD_XM) {
        current_mode = MODE_ALARM;
        return;
    }

    // 预测值超过报警值 → 直接进入危险模式（提前预警）
    if (predicted >= ALARM_THRESHOLD_XM) {
        current_mode = MODE_DANGER;
        return;
    }

    // 根据当前浓度分级
    if (current >= WARNING_THRESHOLD_80) {
        current_mode = MODE_DANGER;
    } else if (current >= WARNING_THRESHOLD_50) {
        current_mode = MODE_ALERT;
    } else if (current >= WARNING_THRESHOLD_30) {
        // 在30%~50%之间，看趋势
        current_mode = (predicted >= WARNING_THRESHOLD_50) ? MODE_ALERT :
MODE_NORMAL;
    } else {
        current_mode = MODE_NORMAL;
    }
}

// =====
// 获取当前模式对应的采样和上传间隔
// =====
static uint16_t get_sample_interval(void)
{
    switch (current_mode) {
        case MODE_DANGER: return SAMPLE_INTERVAL_DANGER;
        case MODE_ALERT: return SAMPLE_INTERVAL_ALERT;
        default: return SAMPLE_INTERVAL_NORMAL;
    }
}

static uint16_t get_upload_interval(void)
{
    switch (current_mode) {

```

```

        case MODE_DANGER:    return 0;    // 立即上传
        case MODE_ALERT:     return UPLOAD_INTERVAL_ALERT;
        default:             return UPLOAD_INTERVAL_NORMAL;
    }
}

// =====
// 设置硬件省电状态
// =====
static void apply_power_mode(void)
{
    // 无论哪种模式，都设置ADC比较器阈值为Xm
    // 休眠期间比较器仍在硬件级监控，一旦超过立即触发中断唤醒MCU
    ADC_SET_COMPARATOR(ALARM_THRESHOLD_XM);

    switch (current_mode) {
        case MODE_DANGER:
        case MODE_ALARM:
            MCU_SLEEP(HW_AWAKE);
            break;
        case MODE_ALERT:
            MCU_SLEEP(HW_SLEEP_LIGHT);
            break;
        default:
            // 深度休眠，仅ADC比较器工作（极低功耗，约1μA）
            MCU_SLEEP(HW_SLEEP_DEEP);
            break;
    }
}

// =====
// ADC比较器中断：休眠期间烟雾浓度超过Xm时硬件自动触发
// 这是满足"3秒响应"的关键——不依赖定时采样，而是硬件实时监控
// =====
volatile bool alarm_triggered = false;

void ADC_COMPARATOR_IRQ(void)
{
    // MCU已被硬件唤醒，立即精确采样确认
    int16_t X_t = ADC_READ();

    if (X_t >= ALARM_THRESHOLD_XM) {
        // 确认超标：立即报警 + 立即上传
        alarm_triggered = true;
        current_mode = MODE_ALARM;
        ALARM_ON();
        // 直接发送，不走FIFO，确保最低延迟
        int16_t alarm_data = X_t;
        WIRELESS_SEND(&alarm_data, sizeof(alarm_data));
    }
    // 如果是毛刺（未超标），不处理，继续休眠
}

// =====
// 主循环
// =====
int main(void)
{

```

```

// 初始化
history.count = 0;
history.index = 0;
upload_buf.count = 0;
current_mode = MODE_NORMAL;

// 设置ADC比较器阈值（休眠时仍可触发唤醒）
ADC_SET_COMPARATOR(WARNING_THRESHOLD_80);

uint32_t tick_count = 0;
uint16_t sample_interval = SAMPLE_INTERVAL_NORMAL;
uint16_t upload_interval = UPLOAD_INTERVAL_NORMAL;

printf("烟雾报警器启动\n");
printf("报警阈值: %d\n", ALARM_THRESHOLD_XM);
printf("初始模式: NORMAL\n\n");

// ===== 主循环（模拟运行120个tick） =====
for (tick_count = 0; tick_count < 120; tick_count++) {

    // --- 0. 检查是否被比较器中断唤醒（硬件路径，<100ms响应） ---
    if (alarm_triggered) {
        // 比较器已触发报警，中断中已完成报警+上传
        printf("[tick %3lu] ★★★ 硬件比较器触发报警 ★★★ 响应延迟 < 100ms\n",
tick_count);
        alarm_triggered = false;
        // 维持报警状态，直到烟雾消散
        continue;
    }

    // --- 1. 软件定时采样（定时器路径） ---
    int16_t X_t = ADC_READ(); // 实际读取烟雾传感器

    // 模拟烟雾浓度变化（测试用）
    // 场景: 0~30秒正常，30~60秒缓慢上升，60~70秒快速上升触发报警
    if (tick_count < 30) {
        X_t = 100 + (int16_t)(tick_count * 2); // 缓慢上升: 100~160
    } else if (tick_count < 60) {
        X_t = 200 + (int16_t)((tick_count - 30) * 15); // 中速上升: 200~650
    } else if (tick_count < 70) {
        X_t = 650 + (int16_t)((tick_count - 60) * 30); // 快速上升: 650~950
    } else {
        X_t = 950 - (int16_t)((tick_count - 70) * 10); // 下降: 950~...
    }

    // --- 2. 存入历史 ---
    history_push(X_t);

    // --- 3. 趋势预测 ---
    int16_t predicted = predict_future();

    // --- 4. 更新运行模式 ---
    update_mode(X_t, predicted);

    // --- 5. 根据模式执行动作 ---
    switch (current_mode) {
        case MODE_ALARM:
            // 报警: 声光报警 + 立即上传

```

```

        ALARM_ON();
        upload_fifo_push(X_t);
        upload_fifo_flush();
        printf("[tick %3lu] X=%4d, pred=%4d | ★★★ ALARM ★★★ 报警! 立即
上传\n",
                tick_count, X_t, predicted);
        break;

    case MODE_DANGER:
        // 危险: 高频采样, 立即上传
        upload_fifo_push(X_t);
        upload_fifo_flush();
        printf("[tick %3lu] X=%4d, pred=%4d | !! DANGER !! 每%s采样, 立即
上传\n",
                tick_count, X_t, predicted, SAMPLE_INTERVAL_DANGER);
        break;

    case MODE_ALERT:
        // 警戒: 攒够数据批量上传
        upload_fifo_push(X_t);
        if (upload_buf.count >= 6) {
            upload_fifo_flush();
        }
        printf("[tick %3lu] X=%4d, pred=%4d | ▲ ALERT 每%s采样, 攒包上传
\n",
                tick_count, X_t, predicted, SAMPLE_INTERVAL_ALERT);
        break;

    case MODE_NORMAL:
        // 正常: 低频采样, 攒够数据批量上传
        upload_fifo_push(X_t);
        if (upload_buf.count >= 6) {
            upload_fifo_flush();
        }
        printf("[tick %3lu] X=%4d, pred=%4d |   NORMAL 每%s采样, 攒包上传
\n",
                tick_count, X_t, predicted, SAMPLE_INTERVAL_NORMAL);
        break;
    }

    // --- 6. 更新采样间隔并配置硬件 ---
    sample_interval = get_sample_interval();
    upload_interval = get_upload_interval();
    RTC_WAKEUP_SET(sample_interval);
    apply_power_mode();
}

// 退出前发送剩余数据
upload_fifo_flush();
printf("\n模拟结束\n");

return 0;
}

```